

Documentación — Desafío 1: Sistema de Clasificación FIFA

Miembros:

1. Descripción general

Este programa resuelve el Desafío 1: Sistema de Clasificación FIFA de la Copa de Algoritmia y Programación UADE 2026.

El objetivo del sistema es procesar los resultados de los partidos de un grupo del Mundial 2026 y determinar qué equipos clasifican a la siguiente ronda. Para eso, el programa lee los resultados desde un archivo de texto, calcula las estadísticas de cada equipo, ordena la tabla de posiciones y muestra los dos clasificados junto con el tercer puesto.

El sistema calcula automáticamente:

puntos obtenidos;

partidos jugados;

partidos ganados;

partidos empatados;

partidos perdidos;

goles a favor;

goles en contra;

diferencia de gol;

posiciones finales;

primer clasificado;

segundo clasificado;

tercer puesto.

La consigna indica que el programa debe procesar los partidos de un grupo, actualizar las estadísticas de cada equipo, ordenar la tabla y determinar los clasificados.

2. Formato del archivo de entrada

El programa recibe los datos desde un archivo llamado:

partidos.txt

La primera línea del archivo indica la cantidad de partidos.

Las siguientes líneas contienen los resultados de cada partido con el siguiente formato:

EquipoLocal EquipoVisitante GolesLocal GolesVisitante

Ejemplo de archivo de entrada:

```
6
ARG BRA 2 1
BRA ESP 1 1
ESP ARG 3 0
ARG JPN 2 0
BRA JPN 2 1
ESP JPN 1 0
```

Según la consigna, cada grupo tiene exactamente 4 equipos, se reciben exactamente 6 partidos y cada equipo juega 3 partidos.

3. Formato de salida

El programa muestra en pantalla los dos equipos clasificados y el tercer puesto.

Formato esperado:

```
Clasificados:
PrimerClasificado
SegundoClasificado
Tercero:
TercerClasificado
```

Ejemplo de salida:

```
Clasificados:
ESP
ARG
Tercero:
BRA
```

Este formato coincide con la salida solicitada por la consigna del desafío.

4. Sistema de puntuación

El programa asigna puntos a cada equipo según el resultado de cada partido.

```
Victoria: 3 puntos
Empate: 1 punto
Derrota: 0 puntos
```

Cuando se procesa un partido, el sistema actualiza las estadísticas de ambos equipos.

Por ejemplo, si el partido es:

```
ARG BRA 2 1
```

Entonces:

ARG suma 3 puntos;

ARG suma 1 partido ganado;

BRA suma 1 partido perdido;

ARG suma 2 goles a favor y 1 gol en contra;

BRA suma 1 gol a favor y 2 goles en contra.

5. Criterios de desempate

La tabla de posiciones se ordena usando los siguientes criterios:

1. Mayor cantidad de puntos.
2. Mayor diferencia de gol.
3. Mayor cantidad de goles a favor.
4. Orden alfabético, en caso de empate absoluto.

La diferencia de gol se calcula así:

Diferencia de gol = goles a favor - goles en contra

Si dos o más equipos tienen la misma cantidad de puntos, la misma diferencia de gol y la misma cantidad de goles a favor, se utiliza el orden alfabético como último criterio. Por ejemplo, ARG queda antes que BRA porque alfabéticamente ARG aparece primero.

6. Estructuras de datos utilizadas

El programa utiliza principalmente un diccionario llamado equipos.

Este diccionario guarda como clave el nombre de cada equipo y como valor otro diccionario con sus estadísticas.

Ejemplo:

```
equipos = {  
  "ARG": {  
    "puntos": 6,  
    "pj": 3,  
    "pg": 2,  
    "pe": 0,  
    "pp": 1,  
    "gf": 4,  
    "gc": 4  
  }  
}
```

Significado de cada dato:

puntos = puntos del equipo
pj = partidos jugados
pg = partidos ganados
pe = partidos empatados
pp = partidos perdidos
gf = goles a favor
gc = goles en contra

También se utiliza una lista llamada ranking.

Esta lista permite ordenar a los equipos según los criterios de clasificación.

Cada equipo dentro del ranking se guarda con esta estructura:

[nombre, puntos, diferencia_gol, goles_favor, goles_contra, partidos_jugados, ganados, empatados, perdidos]

Esta organización permite manejar los datos de forma clara y facilita el cálculo de la tabla final.

7. Funciones principales del programa

crear_equipo()

Crea la estructura inicial de un equipo nuevo.

Todas las estadísticas comienzan en cero.

Se utiliza cuando aparece un equipo que todavía no estaba registrado en el diccionario principal.

agregar_equipo_si_no_existe(equipos, nombre)

Verifica si un equipo ya existe dentro del diccionario equipos.

Si el equipo no existe, lo crea usando la función crear_equipo().

Esto evita tener que cargar manualmente los equipos antes de procesar los partidos.

goles_validos(goles)

Valida que la cantidad de goles esté dentro del rango permitido.

Según la consigna, los goles deben ser enteros entre 0 y 20.

La función devuelve True si los goles son válidos y False si no lo son.

grupo_valido(equipos)

Valida que el grupo cumpla con las condiciones principales del desafío.

Controla que:

haya exactamente 4 equipos;

cada equipo tenga exactamente 3 partidos jugados.

Si alguna de estas condiciones no se cumple, el programa muestra un mensaje de error y no continúa con el ranking.

`procesar_partido(equipos, local, visitante, goles_local, goles_visitante)`

Procesa un partido completo.

Esta función actualiza las estadísticas del equipo local y del equipo visitante.

Actualiza:

partidos jugados;

goles a favor;

goles en contra;

puntos;

partidos ganados;

partidos empatados;

partidos perdidos.

También contiene la lógica para determinar si ganó el local, ganó el visitante o si el partido terminó empatado.

`leer_partidos_desde_archivo(nombre_archivo)`

Abre el archivo de entrada y lee los partidos.

Primero lee la cantidad de partidos. Luego, mediante un ciclo for, lee cada línea con los datos del partido.

Por cada partido leído, separa los datos usando `split()` y luego llama a la función `procesar_partido()`.

También incluye validaciones para verificar que los datos del archivo sean correctos.

Al finalizar, devuelve el diccionario `equipos` con todas las estadísticas actualizadas.

`calcular_diferencia_gol(datos_equipo)`

Calcula la diferencia de gol de un equipo.

La fórmula utilizada es:

goles a favor - goles en contra

Ejemplo:

GF = 5

GC = 1

DG = 5 - 1 = 4

`criterio_ordenamiento(equipo)`

Define los criterios para ordenar la tabla de posiciones.

El orden aplicado es:

puntos

diferencia de gol

goles a favor

nombre

Los puntos, la diferencia de gol y los goles a favor se usan en negativo porque la función `sort()` ordena de menor a mayor. Al poner esos valores en negativo, los equipos con mejores estadísticas quedan primero.

El nombre del equipo se mantiene sin negativo para que Python lo ordene alfabéticamente en caso de empate absoluto.

`obtener_ranking(equipos)`

Convierte el diccionario `equipos` en una lista llamada `ranking`.

Cada equipo se agrega al `ranking` con sus estadísticas principales.

Luego, la lista se ordena usando la función `criterio_ordenamiento()`.

Al final, devuelve el `ranking` ordenado desde el mejor equipo hasta el peor.

`mostrar_clasificados(ranking)`

Muestra la salida final del programa.

Como el `ranking` ya está ordenado:

`ranking[0][0]` corresponde al primer clasificado;

ranking[1][0] corresponde al segundo clasificado;

ranking[2][0] corresponde al tercer puesto.

La salida se muestra con el formato pedido por la consigna:

Clasificados:

ESP

ARG

Tercero:

BRA

8. Validaciones implementadas

El programa incluye validaciones para evitar errores en los datos de entrada.

Se valida que:

la cantidad de partidos sea exactamente 6;

cada línea del archivo tenga exactamente 4 datos;

un equipo no juegue contra sí mismo;

los goles sean números enteros;

los goles estén entre 0 y 20;

el grupo tenga exactamente 4 equipos;

cada equipo haya jugado exactamente 3 partidos.

Estas validaciones ayudan a que el programa sea más robusto ante datos incorrectos o casos límite.

9. Casos de prueba

Caso 1: ejemplo principal

Entrada:

6

ARG BRA 2 1

BRA ESP 1 1

ESP ARG 3 0

ARG JPN 2 0

BRA JPN 2 1

ESP JPN 1 0

Salida esperada:

Clasificados:
ESP
ARG
Tercero:
BRA

Este caso verifica el funcionamiento general del sistema.

Caso 2: desempate por goles a favor

Entrada:

6
ARG BRA 1 0
BRA ESP 2 0
ESP ARG 3 1
ARG JPN 2 0
BRA JPN 1 0
ESP JPN 2 1

Salida esperada:

Clasificados:
ESP
ARG
Tercero:
BRA

En este caso, ARG, BRA y ESP terminan con la misma cantidad de puntos y la misma diferencia de gol. El desempate se define por mayor cantidad de goles a favor.

Caso 3: empate absoluto

Entrada:

6
ARG BRA 1 1
ARG ESP 1 1
ARG JPN 1 1
BRA ESP 1 1
BRA JPN 1 1
ESP JPN 1 1

Salida esperada:

Clasificados:
ARG
BRA
Tercero:
ESP

En este caso, todos los equipos empatan en puntos, diferencia de gol y goles a favor. Por eso, se aplica el orden alfabético.

10. Modo de uso

1. Abrir el proyecto en Visual Studio Code.
2. Crear o verificar el archivo partidos.txt.
3. Escribir los resultados respetando el formato indicado.
4. Ejecutar el archivo Python principal.
5. Leer en pantalla los equipos clasificados y el tercer puesto.

Ejemplo de ejecución:

```
python3 desafio1_ranking_fifa.py
```

En este proyecto, el archivo de entrada se lee desde la ruta:

Desafio 1/codigo/partidos.txt

Por lo tanto, el archivo partidos.txt debe estar ubicado en esa carpeta o se debe modificar la ruta dentro del código.

11. Conclusión

El programa cumple con los requisitos principales del desafío. Procesa los partidos de un grupo, actualiza las estadísticas de cada equipo, ordena la tabla de posiciones aplicando los criterios de desempate y muestra los equipos clasificados en el formato solicitado.

Además, utiliza funciones, diccionarios, listas, validaciones y ordenamiento con múltiples criterios, lo que permite que el código sea claro, organizado y robusto.